

A05_ CVE-2022-27195

Here is a snippet of code written in java. I want to check for any potential security vulnerabilities.

```
package hudson.plugins.parameterizedtrigger;

import hudson.EnvVars;
import hudson.model.InvisibleAction;

/**
 * @author <a
 href="mailto:nicolas.delooof@gmail.com">Nicolas De
 Loof</a>
 */
public class CapturedEnvironmentAction extends
InvisibleAction {

    private final EnvVars env;

    public CapturedEnvironmentAction(EnvVars env) {
        this.env = env;
    }

    public EnvVars getCapturedEnvironment() {
        return env;
    }
}
```

Let's review the provided Java code for potential security vulnerabilities. Here are some aspects to watch for:

Security Review of Code:

java

```
package hudson.plugins.parameterizedtrigger;

import hudson.EnvVars;
```

```
import hudson.model.InvisibleAction;

public class CapturedEnvironmentAction extends InvisibleAction {

    private final EnvVars env;

    public CapturedEnvironmentAction(EnvVars env) {
        this.env = env;
    }

    public EnvVars getCapturedEnvironment() {
        return env;
    }
}
```

Potential Security Issues:

1. Exposure of Environment Variables

- **Risk:** This class stores and exposes environment variables through `EnvVars env`. Environment variables may contain **sensitive information**, such as API keys, credentials, or internal system details.
- **Mitigation:**
 - Ensure only non-sensitive environment variables are captured and used.
 - Apply strict access control to prevent unintentional exposure of the environment.
 - Mask or filter sensitive variables (e.g., `PASSWORD`, `API_KEY`) before returning them via `getCapturedEnvironment()`.

2. Lack of Input Validation

- **Risk:** If the `EnvVars env` input comes from an external or untrusted source, it could lead to **malicious payload injection**.
- **Mitigation:** Validate and sanitize the content of the `EnvVars` before storing it, especially if some environment values are later used in **system commands** or output to logs.

3. Invisible Action Handling in Jenkins

- **Risk:** Since this code is part of Jenkins plugins (as inferred from the package), the `InvisibleAction` class means this action is **invisible in the user interface** but still executable in the build process. This may create **audit blind spots**.
- **Mitigation:** Be cautious about how and where this action is used. Ensure that:
 - Only trusted code triggers this action.
 - Implement logging and audit mechanisms to monitor its use, if applicable.

4. Immutable Object Risk

- **Risk:** `EnvVars` might be mutable, meaning a malicious actor could **alter the values** in `env` after the object has been passed to the constructor.
- **Mitigation:**
 - Consider **defensive copying** to create a snapshot of the environment variables instead of storing the original reference.

java

```
this.env = new EnvVars(env);
```

- Alternatively, return a copy of the environment from `getCapturedEnvironment()` instead of the original object:

java

```
public EnvVars getCapturedEnvironment() {  
    return new EnvVars(env);  
}
```

Suggestions:

1. Audit Sensitive Data Handling:

- Check whether `CapturedEnvironmentAction` is used in a way that could unintentionally expose sensitive data in Jenkins logs, reports, or other external outputs.

2. Access Control:

- Ensure this class is only accessible and used in secure contexts with the appropriate permissions.

Revised Code for Better Security:

```
java

package hudson.plugins.parameterizedtrigger;

import hudson.EnvVars;
import hudson.model.InvisibleAction;

public class CapturedEnvironmentAction extends InvisibleAction {

    private final EnvVars env;

    public CapturedEnvironmentAction(EnvVars env) {
        // Create a defensive copy of the environment variables
        this.env = new EnvVars(env);
    }

    public EnvVars getCapturedEnvironment() {
        // Return a new copy to prevent modification of the original environment
        return new EnvVars(env);
    }
}
```

Summary:

- The primary concerns in this code are **exposure of sensitive data**, **mutability risks**, and **auditability** within the Jenkins ecosystem.
- Implementing **defensive copying** and **controlled access** can mitigate some risks.

- Additionally, carefully review how this action is invoked and ensure sensitive variables are filtered if needed.

Let me know if you need further analysis or suggestions!

Provide the exact part of the code along with line numbers that are vulnerable or have security issues.

Here's a **line-by-line analysis** of the security concerns identified in the provided Java code, with specific vulnerabilities highlighted:

Annotated Code with Line Numbers and Vulnerabilities

java

```
1  package hudson.plugins.parameterizedtrigger;

2  import hudson.EnvVars;
3  import hudson.model.InvisibleAction;

4  public class CapturedEnvironmentAction extends InvisibleAction {

5      private final EnvVars env; // [Potential Issue] See Line 12

6      public CapturedEnvironmentAction(EnvVars env) {
7          this.env = env; // [Line 7: Issue - Lack of Defensive Copying]
                          // Risk: If `env` is mutable, external changes could
                          // compromise integrity.
                          // Fix: Create a defensive copy to prevent unintended
                          // modifications.
                          // Suggested Fix: this.env = new EnvVars(env);
8      }

9      public EnvVars getCapturedEnvironment() {
10         return env; // [Line 10: Issue - Exposing Internal State]
                     // Risk: Returning the internal `EnvVars` directly can allow
                     // unintended modification.
                     // Fix: Return a new copy to prevent external changes.
                     // Suggested Fix: return new EnvVars(env);
```

```
11     }  
12 }
```

Detailed Security Issues by Line:

1. Line 7 - Lack of Defensive Copying:

```
java  
  
this.env = env;
```

- **Vulnerability:**

- The code assigns the incoming `EnvVars` object directly to the internal `env` field.
- If `EnvVars` is mutable (as it likely is), any changes to the passed object outside this class will also affect the internal state of `CapturedEnvironmentAction`.

- **Fix:**

Replace with:

```
java  
  
this.env = new EnvVars(env);
```

2. Line 10 - Exposing Internal State:

```
java  
  
return env;
```

- **Vulnerability:**

- The method returns the **original** `EnvVars` instance, which may allow modification by any caller.
- This can lead to unintended changes to the environment variables stored within the object.

- **Fix:**

Replace with:

```
java

return new EnvVars(env);
```

3. Line 5 - Sensitive Data Handling (Potential Issue):

```
java

private final EnvVars env;
```

- **Risk:**

- Environment variables can contain sensitive data, such as API keys, passwords, or secrets. Exposing them through `getCapturedEnvironment()` might leak sensitive information.

- **Recommendation:**

- Add filtering logic to ensure that sensitive variables are not included or logged unintentionally.

Revised Secure Code:

Here is the corrected version with the identified vulnerabilities mitigated:

```
java

package hudson.plugins.parameterizedtrigger;

import hudson.EnvVars;
import hudson.model.InvisibleAction;

public class CapturedEnvironmentAction extends InvisibleAction {

    private final EnvVars env;

    public CapturedEnvironmentAction(EnvVars env) {
        // Defensive copying to avoid external changes
    }
}
```

```
        this.env = new EnvVars(env);
    }

    public EnvVars getCapturedEnvironment() {
        // Return a copy to prevent external modification of the internal state
        return new EnvVars(env);
    }
}
```

Summary of Vulnerabilities:

1. **Line 7:** Lack of **defensive copying** when storing the `EnvVars` object.
2. **Line 10:** **Exposing internal state** directly, risking unintended modification.
3. **Line 5:** Potential **sensitive data exposure** via environment variables (depends on use).

By applying defensive programming practices, these vulnerabilities can be mitigated.

Please analyze the code for common security vulnerabilities if present and map them to OWASP Top 10 Web Application Security Risk category/categories.

Here's a detailed analysis of the provided Java code, focusing on security vulnerabilities and their mapping to the **OWASP Top 10 Web Application Security Risks**:

Code Recap

```
java

package hudson.plugins.parameterizedtrigger;

import hudson.EnvVars;
import hudson.model.InvisibleAction;

public class CapturedEnvironmentAction extends InvisibleAction {
```

```

private final EnvVars env;

public CapturedEnvironmentAction(EnvVars env) {
    this.env = env; // Potential Vulnerability: Lack of Defensive Copying
}

public EnvVars getCapturedEnvironment() {
    return env; // Potential Vulnerability: Exposing Internal State
}
}

```

Potential Security Vulnerabilities and OWASP Top 10 Mapping

Line Number	Vulnerability	Description	OWASP Top 10 Mapping
Line 7	Lack of Defensive Copying	Assigning the external <code>EnvVars</code> object directly without defensive copying allows unintended modification of internal state.	A04: Insecure Design – The design does not account for potential side effects caused by mutable objects passed into the constructor. This can lead to an inconsistent or corrupted state.
Line 10	Exposing Internal State	Returning the original <code>EnvVars</code> object makes it possible for an external actor to modify the internal state.	A01: Broken Access Control – The method provides unintended access to sensitive data by returning the mutable environment variable object.
Line 5, 10	Potential Data Exposure	The environment variables may contain sensitive information (e.g., API keys, passwords), which could be exposed if accessed inappropriately.	A02: Cryptographic Failures – Exposing environment variables could lead to the accidental disclosure of sensitive information.
Line 5, 10	Insufficient Data Sanitization	If untrusted data is stored in the environment variables, malicious input might flow into other parts of the system, especially if these variables are logged or used in commands.	A06: Vulnerable and Outdated Components – If the class relies on <code>EnvVars</code> from untrusted sources, it could introduce vulnerabilities through

Line Number	Vulnerability	Description	OWASP Top 10 Mapping
			insufficient validation or sanitization.

Detailed OWASP Top 10 Mapping and Risk Explanation

1. A01: Broken Access Control

- *Risk:* The `getCapturedEnvironment()` method provides access to internal `EnvVars` without control or restrictions. This allows any caller to access and modify environment variables, which could contain sensitive information.
- *Impact:* It can lead to unintended privilege escalation or data leakage. If these environment variables are improperly accessed, it could give attackers unauthorized access to sensitive configurations.

2. A02: Cryptographic Failures (Sensitive Data Exposure)

- *Risk:* If environment variables contain sensitive information (like passwords or API keys) and are exposed through the `getCapturedEnvironment()` method, it could lead to sensitive data leakage.
- *Impact:* Exposing these variables could compromise accounts, APIs, or systems that rely on the confidentiality of such data.

3. A04: Insecure Design

- *Risk:* The lack of **defensive copying** in the constructor makes the class vulnerable to accidental or intentional changes from outside code. This design flaw could lead to state corruption or unintended behavior.
- *Impact:* An external actor could modify the internal state of `CapturedEnvironmentAction` without notice, leading to unexpected behavior in the build process.

4. A06: Vulnerable and Outdated Components

- *Risk:* The code relies on `EnvVars` without sanitizing or validating its content. If the environment variables are user-controlled or passed from untrusted sources, it might introduce vulnerabilities through malicious input.

- *Impact:* If these environment variables are later used in **system commands** or **logged** without proper sanitization, it can lead to security issues like **command injection** or **log injection**.
-

Suggested Mitigations for Identified Issues

1. Defensive Copying (Line 7)

- Replace:

```
java  
  
this.env = env;
```

- With:

```
java  
  
this.env = new EnvVars(env);
```

2. Return a Copy Instead of the Original (Line 10)

- Replace:

```
java  
  
return env;
```

- With:

```
java  
  
return new EnvVars(env);
```

3. Filter Sensitive Variables (Line 5, 10)

- Implement logic to **filter out sensitive variables** from the environment before returning it.

```
java  
  
public EnvVars getCapturedEnvironment() {  
    EnvVars safeEnv = new EnvVars(env);
```

```
safeEnv.remove("API_KEY");  
safeEnv.remove("PASSWORD");  
return safeEnv;  
}
```

4. Restrict Access Control to `getCapturedEnvironment()` Method

- Add access control mechanisms to ensure this method is only used by **authorized callers**.

```
java  
  
@Restricted(NoExternalUse.class) // Jenkins-specific annotation to restrict  
access  
public EnvVars getCapturedEnvironment() {  
    return new EnvVars(env);  
}
```

Conclusion

This code exhibits **design flaws** that align with several OWASP Top 10 risks, including **A01: Broken Access Control**, **A02: Cryptographic Failures**, **A04: Insecure Design**, and **A06: Vulnerable and Outdated Components**. By implementing **defensive copying**, **restricting access**, and **filtering sensitive data**, these risks can be mitigated effectively.

Let me know if you need further assistance or more detailed examples!

For each identified vulnerability, provide an exploit example that an attacker could use. Include a step-by-step explanation of how the exploit works, the necessary conditions or setup, any potential limitations or obstacles an attacker might face, and the likelihood of this attack happening. Ensure the exploits are specific to the vulnerability type provided.

Here is a detailed breakdown of potential exploits for each identified vulnerability, including **step-by-step explanations**, **necessary conditions**, **limitations**, and **likelihood of attack**:

1. Lack of Defensive Copying (Line 7)

Vulnerability Recap:

The constructor directly assigns the passed `EnvVars` object reference to the private field without creating a defensive copy. If the `EnvVars` object is mutable, an attacker could modify it after the `CapturedEnvironmentAction` object is created, leading to an inconsistent or malicious state.

Exploit Example:

1. Setup:

- An attacker has access to the `EnvVars` object (e.g., passed as a parameter to the constructor).
- The attacker provides a crafted `EnvVars` object to the constructor.

2. Step-by-Step Exploit:

- The attacker creates an `EnvVars` object with legitimate data but also includes references to mutable fields or sensitive data.
- The attacker modifies the `EnvVars` object after the `CapturedEnvironmentAction` object is constructed, injecting malicious data or overwriting critical variables.

3. Example Code:

```
java

EnvVars maliciousEnv = new EnvVars();
maliciousEnv.put("VALID_KEY", "valid_value");
CapturedEnvironmentAction action = new CapturedEnvironmentAction(maliciousEnv);

// Later, the attacker modifies the original object.
maliciousEnv.put("VALID_KEY", "malicious_value");
```

4. Impact:

- The internal state of `CapturedEnvironmentAction` is corrupted or maliciously altered.

Necessary Conditions:

- The attacker must have access to supply the `EnvVars` object to the constructor.

Limitations:

- This attack is limited to scenarios where untrusted sources can pass the `EnvVars` object.

Likelihood:

- **Moderate** – The likelihood increases in multi-user systems or when external inputs are used to initialize `EnvVars`.
-

2. Exposing Internal State (Line 10)

Vulnerability Recap:

The `getCapturedEnvironment()` method returns the original `EnvVars` object, allowing a caller to modify it directly.

Exploit Example:

1. Setup:

- The attacker obtains a reference to the `CapturedEnvironmentAction` object and calls the `getCapturedEnvironment()` method.

2. Step-by-Step Exploit:

- The attacker retrieves the `EnvVars` object using `getCapturedEnvironment()`.
- The attacker modifies the environment variables directly to introduce malicious behavior or cause system misconfiguration.

3. Example Code:

```
java

CapturedEnvironmentAction action = new CapturedEnvironmentAction(envVars);
EnvVars captured = action.getCapturedEnvironment();

// The attacker modifies the environment directly.
captured.put("API_KEY", "malicious_key");
captured.put("PATH", "/malicious/path");
```

4. Impact:

- The attacker compromises environment-dependent functionality, such as build scripts or external integrations.

Necessary Conditions:

- The attacker must have access to call `getCapturedEnvironment()`.

Limitations:

- This attack is limited to systems where the returned `EnvVars` object affects sensitive operations.

Likelihood:

- **High** – Systems relying on unprotected environment variables are vulnerable in multi-user environments or plugins.
-

3. Sensitive Data Exposure (Lines 5, 10)

Vulnerability Recap:

The `CapturedEnvironmentAction` class potentially stores and exposes sensitive environment variables, such as API keys or passwords.

Exploit Example:

1. Setup:

- The system is configured to store sensitive credentials in environment variables.
- The attacker gains access to call `getCapturedEnvironment()`.

2. Step-by-Step Exploit:

- The attacker retrieves the `EnvVars` object using `getCapturedEnvironment()`.
- The attacker searches for sensitive variables such as `API_KEY`, `DB_PASSWORD`, or `SECRET`.
- The attacker uses these secrets for malicious purposes (e.g., accessing a database or API).

3. Example Code:

```
java

EnvVars captured = action.getCapturedEnvironment();

// Extract sensitive data
```

```
String apiKey = captured.get("API_KEY");
String dbPassword = captured.get("DB_PASSWORD");

System.out.println("API Key: " + apiKey);
System.out.println("Database Password: " + dbPassword);
```

4. Impact:

- Compromised secrets can lead to API abuse, data theft, or privilege escalation.

Necessary Conditions:

- The environment variables must store sensitive information.
- The attacker must have access to the `CapturedEnvironmentAction` object.

Limitations:

- Sensitive variables must exist in the `EnvVars` object.

Likelihood:

- **High** – Sensitive data is often stored in environment variables, making this a common issue.

4. Insufficient Data Sanitization (Lines 5, 10)

Vulnerability Recap:

If environment variables contain untrusted input, they may be used in a way that leads to **command injection**, **log injection**, or other forms of exploitation.

Exploit Example:

1. Setup:

- The system passes untrusted environment variables into `CapturedEnvironmentAction`.
- Downstream processes rely on these environment variables, such as in shell commands or logs.

2. Step-by-Step Exploit:

- The attacker includes a malicious payload in an environment variable (e.g., injecting shell commands).
- The malicious payload executes during downstream use of `CapturedEnvironmentAction`.

3. Example Payload in Environment Variable:

```
bash
```

```
ENV_VAR="malicious_value; rm -rf /important/data"
```

4. Impact Example (Command Injection):

```
java
```

```
String command = "echo " + captured.get("ENV_VAR");  
Runtime.getRuntime().exec(command);  
// Executes: echo malicious_value; rm -rf /important/data
```

5. Impact Example (Log Injection):

```
java
```

```
System.out.println("Captured Env: " + captured.get("ENV_VAR"));  
// Log file now contains:  
// Captured Env: malicious_value  
// rm -rf /important/data
```

Necessary Conditions:

- The attacker controls or influences the content of environment variables.
- The environment variables are used unsafely in downstream processes.

Limitations:

- Requires downstream misuse of untrusted environment variables.

Likelihood:

- **Moderate** – Relies on specific misuse of untrusted data.

Summary of Exploit Likelihoods

Vulnerability	Exploit Likelihood	Reason
Lack of Defensive Copying	Moderate	Common in systems accepting untrusted inputs, especially in Jenkins plugins.
Exposing Internal State	High	Likely to occur in multi-user or plugin ecosystems.
Sensitive Data Exposure	High	Many systems store sensitive data in environment variables, making exposure critical.
Insufficient Data Sanitization	Moderate	Relies on specific downstream misuse of environment variables, but remains a common risk.

By mitigating these vulnerabilities through **defensive copying**, **access control**, and **input sanitization**, the risks of these exploits can be significantly reduced. Let me know if you'd like more mitigation examples!

2/2